

**LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 12**



Disusun Oleh:

Moh Ilham Ramadhan Kurniawan 105224014

**PROGRAM STUDI ILMU KOMPUTER
FAKULTAS SAINS DAN KOMPUTER
UNIVERSITAS PERTAMINA
2026**

I. Pendahuluan

Perkembangan sistem informasi akademik di lingkungan perguruan tinggi menuntut adanya perangkat lunak yang mampu menangani berbagai proses akademik secara cepat, efisien, dan mudah dikembangkan. Salah satu sistem yang memiliki peran penting adalah Sistem Informasi Akademik (SIKAD) yang digunakan untuk mengelola proses pengisian Kartu Rencana Studi (KRS), pengelolaan mata kuliah, perhitungan biaya pendidikan, serta penyimpanan data akademik mahasiswa.

Pada studi kasus ini ditemukan berbagai permasalahan pada sistem SIKAD yang telah digunakan sebelumnya. Seluruh proses bisnis ditempatkan pada satu class besar bernama SistemKRSManager sehingga menyebabkan tingginya ketergantungan antar komponen. Akibatnya, perubahan kecil pada salah satu fitur dapat menimbulkan kerusakan pada fitur lainnya. Selain itu, sistem juga sulit dikembangkan karena tidak menerapkan prinsip-prinsip perancangan perangkat lunak yang baik.

Untuk mengatasi permasalahan tersebut dilakukan proses refactoring dengan menerapkan prinsip SOLID yang terdiri dari Single Responsibility Principle (SRP), Open Closed Principle (OCP), Liskov Substitution Principle (LSP), Interface Segregation Principle (ISP), dan Dependency Inversion Principle (DIP). Penerapan prinsip-prinsip tersebut bertujuan untuk menghasilkan sistem yang lebih fleksibel, mudah dipelihara, serta mampu mengakomodasi kebutuhan baru seperti migrasi database ke Cloud NoSQL dan penambahan skema perhitungan UKT untuk program MBKM.

Melalui implementasi ini diharapkan dapat memberikan pemahaman mengenai penerapan prinsip SOLID dalam menyelesaikan permasalahan desain perangkat lunak serta meningkatkan kualitas arsitektur sistem agar lebih siap menghadapi perubahan kebutuhan di masa mendatang.

II. Variabel

No	Nama Variabel	Tipe Data	Fungsi
1	nim	String	Menyimpan Nomor Induk Mahasiswa
2	totalSKS	int	Menyimpan jumlah SKS yang diambil mahasiswa
3	strategy	UKTStrategy	Menyimpan strategi perhitungan UKT yang digunakan
4	validator	PrerequisiteValidator	Digunakan untuk melakukan validasi prasyarat mata kuliah
5	repository	KRSRepository	Digunakan untuk menyimpan data KRS ke database
6	pdfGenerator	PDFGenerator	Digunakan untuk membuat dokumen PDF KRS
7	nama	String	Menyimpan nama mata kuliah
8	sks	int	Menyimpan jumlah SKS mata kuliah
9	krs	KRS	Menyimpan objek KRS mahasiswa
10	calculator	UKTCalculator	Digunakan untuk menghitung UKT mahasiswa
11	ukt	double	Menyimpan hasil perhitungan UKT
12	service	KRSService	Menjalankan proses pengelolaan KRS
13	repository	KRSRepository	Menyimpan implementasi database yang digunakan
14	pdfGenerator	PDFGenerator	Menghasilkan dokumen KRS dalam bentuk PDF
15	validator	PrerequisiteValidator	Memvalidasi prasyarat pengambilan mata kuliah

III. Dokumentasi dan Pembahasan Code

Program ini merupakan hasil perancangan ulang (refactoring) sistem SIAKAD yang sebelumnya memiliki berbagai permasalahan arsitektur seperti God Class, Tight Coupling, serta pelanggaran terhadap prinsip-prinsip SOLID. Pada sistem lama, seluruh proses pengelolaan KRS, validasi prasyarat, perhitungan UKT, pembuatan PDF, dan penyimpanan data ditempatkan dalam satu class besar bernama SistemKRSManager sehingga perubahan kecil pada satu fitur berpotensi merusak fitur lainnya.

Untuk mengatasi permasalahan tersebut, sistem dirancang ulang dengan memisahkan tanggung jawab ke dalam beberapa class yang lebih spesifik. Perhitungan UKT menggunakan Strategy Pattern sehingga penambahan jalur mahasiswa baru dapat dilakukan tanpa mengubah kode yang sudah ada. Mekanisme penyimpanan data menggunakan abstraction melalui interface KRSRepository sehingga sistem dapat berpindah dari MySQL ke NoSQL tanpa memengaruhi class lain. Selain itu, interface mata kuliah dipisahkan berdasarkan kebutuhan sehingga setiap jenis mata kuliah hanya mengimplementasikan method yang benar-benar diperlukan.

Dengan desain ini, sistem menjadi lebih mudah dipelihara, mudah dikembangkan, serta lebih fleksibel dalam menghadapi perubahan kebutuhan di masa mendatang.

KRS.java

```
1 package model;
2
3 public class KRS {
4     private String nim;
5     private int totalSKS;
6
7     public KRS(String nim, int totalSKS) {
8         this.nim = nim;
9         this.totalSKS = totalSKS;
10    }
11
12    public String getNim() {
13        return nim;
14    }
15
16    public int getTotalSKS() {
17        return totalSKS;
18    }
19 }
```

Class KRS digunakan sebagai model data yang merepresentasikan informasi Kartu Rencana Studi mahasiswa.

Atribut nim digunakan untuk menyimpan Nomor Induk Mahasiswa.

Atribut totalSKS digunakan untuk menyimpan jumlah SKS yang diambil mahasiswa.

Constructor KRS() digunakan untuk menginisialisasi data KRS ketika objek baru dibuat.

Method getNim() digunakan untuk mengambil nilai NIM mahasiswa.

Method getTotalSKS() digunakan untuk mengambil jumlah total SKS yang diambil mahasiswa.

PrerequisiteValidator.java

```
1 package service;
2
3 public class PrerequisiteValidator {
4
5     public boolean validate(String nim) {
6         System.out.println("Validasi prasyarat mahasiswa " + nim);
7         return true;
8     }
9 }
```

Class PrerequisiteValidator bertanggung jawab melakukan validasi prasyarat pengambilan mata kuliah.

Class ini dibuat terpisah agar hanya memiliki satu tanggung jawab sesuai prinsip Single Responsibility Principle (SRP).

Method validate() digunakan untuk memeriksa apakah mahasiswa memenuhi syarat untuk melakukan pengambilan mata kuliah.

Method mengembalikan nilai boolean yang menunjukkan hasil validasi.

PDFGenerator.java

```
1 package report;
2
3 import model.KRS;
4
5 public class PDFGenerator {
6
7     public void generate(KRS krs) {
8         System.out.println(
9             "Membuat PDF KRS untuk " + krs.getNim()
10        );
11     }
12 }
```

Class PDFGenerator bertanggung jawab untuk menghasilkan dokumen KRS dalam bentuk PDF.

Pemisahan proses pembuatan PDF ke dalam class tersendiri bertujuan agar perubahan format dokumen tidak memengaruhi proses bisnis lainnya.

Method generate() digunakan untuk membuat dokumen PDF berdasarkan data KRS yang diterima.

Penerapan ini sesuai dengan prinsip Single Responsibility Principle (SRP).

KRSRepository.java

```

1 package repository;
2
3 import model.KRS;
4
5 public interface KRSRepository {
6
7     void save(KRS krs);
8
9 }

```

KRSRepository merupakan interface yang mendefinisikan kontrak penyimpanan data KRS.

Interface ini digunakan sebagai abstraksi agar class lain tidak bergantung langsung pada jenis database tertentu.

Method save() digunakan untuk menyimpan data KRS.

Penerapan interface ini mendukung prinsip Dependency Inversion Principle (DIP).

MySQLRepository.java

```

1 package repository;
2
3 import model.KRS;
4
5 public class MySQLRepository implements KRSRepository {
6
7     @Override
8     public void save(KRS krs) {
9         System.out.println("Data disimpan ke MySQL");
10    }
11 }

```

Class MySQLRepository merupakan implementasi dari interface KRSRepository.

Class ini digunakan untuk menyimpan data KRS ke database MySQL.

Method save() berisi proses penyimpanan data ke sistem basis data MySQL.

Karena menggunakan interface, implementasi ini dapat diganti tanpa mengubah class lain.

NoSQLRepository.java

```

1 package repository;
2
3 import model.KRS;
4
5 public class NoSQLRepository implements KRSRepository {
6
7     @Override
8     public void save(KRS krs) {
9         System.out.println("Data disimpan ke Cloud NoSQL");
10    }
11 }

```

Class NoSQLRepository merupakan implementasi lain dari interface KRSRepository.

Class ini digunakan untuk menyimpan data KRS ke database Cloud NoSQL.

Method save() digunakan untuk melakukan penyimpanan data ke sistem NoSQL. Penerapan ini memungkinkan migrasi database dari MySQL ke NoSQL secara mudah sesuai kebutuhan kampus.

UKTStrategy.java

```
src > strategy > J UKTStrategy.java > UKTStrategy
1  package strategy;
2
3  public interface UKTStrategy {
4
5      double calculate(int sks);
6
7  }
```

UKTStrategy merupakan interface yang digunakan untuk mendefinisikan strategi perhitungan UKT.

Interface ini dibuat agar setiap jalur mahasiswa memiliki algoritma perhitungan UKT yang berbeda.

Method calculate() digunakan untuk menghitung biaya UKT berdasarkan jumlah SKS yang diambil.

Penerapan interface ini mendukung Open Closed Principle (OCP).

RegulerStrategy.java

```
1  package strategy;
2
3  public class RegulerStrategy implements UKTStrategy {
4
5      @Override
6      public double calculate(int sks) {
7          return sks * 150000;
8      }
9  }
```

Class RegulerStrategy merupakan implementasi dari interface UKTStrategy.

Class ini digunakan untuk menghitung UKT mahasiswa jalur reguler.

Method calculate() berisi rumus perhitungan UKT yang berlaku untuk mahasiswa reguler.

KaryawanStrategy.java

```
src > strategy > KaryawanStrategy.java > KaryawanStrategy
1  package strategy;
2
3  public class KaryawanStrategy implements UKTStrategy {
4
5      @Override
6      public double calculate(int sks) {
7          return sks * 200000;
8      }
9  }
```

Class KaryawanStrategy merupakan implementasi dari interface UKTStrategy.

Class ini digunakan untuk menghitung UKT mahasiswa jalur karyawan.

Method calculate() berisi rumus perhitungan UKT khusus mahasiswa karyawan.

InternasionalStrategy.java

```
src > strategy > J InternasionalStrategy.java > InternasionalStrategy
1  package strategy;
2
3  public class InternasionalStrategy implements UKTStrategy {
4
5      @Override
6      public double calculate(int sks) {
7          return sks * 300000;
8      }
9  }
```

Class InternasionalStrategy merupakan implementasi dari interface UKTStrategy. Class ini digunakan untuk menghitung UKT mahasiswa internasional. Method calculate() berisi algoritma perhitungan biaya UKT untuk mahasiswa internasional.

MBKMStrategy.java

```
1  package strategy;
2
3  public class MBKMStrategy implements UKTStrategy {
4
5      @Override
6      public double calculate(int sks) {
7          return sks * 100000;
8      }
9  }
```

Class MBKMStrategy merupakan implementasi dari interface UKTStrategy. Class ini digunakan untuk menghitung UKT mahasiswa yang mengikuti program MBKM.

Penambahan class ini dapat dilakukan tanpa mengubah class lain sehingga memenuhi prinsip Open Closed Principle (OCP).

Method calculate() digunakan untuk menghitung UKT berdasarkan aturan program MBKM.

UKTCalculator.java

```
1  package service;
2
3  import strategy.UKTStrategy;
4
5  public class UKTCalculator {
6
7      private UKTStrategy strategy;
8
9      public UKTCalculator(UKTStrategy strategy) {
10         this.strategy = strategy;
11     }
12
13     public double calculate(int sks) {
14         return strategy.calculate(sks);
15     }
16 }
```

Class UKTCalculator bertugas melakukan proses perhitungan UKT menggunakan strategi yang dipilih.

Atribut strategy digunakan untuk menyimpan objek implementasi UKTStrategy.

Constructor UKTCalculator() digunakan untuk menerima strategi perhitungan UKT yang akan digunakan.

Method calculate() digunakan untuk memanggil algoritma perhitungan yang sesuai dengan jenis mahasiswa.

Class ini menerapkan konsep Strategy Pattern.

MataKuliah.java

```
src > matakuliah > J MataKuliah.java > MataKuliah
1  package matakuliah;
2
3  public interface MataKuliah {
4
5      String getNama();
6
7      int getSKS();
8  }
```

MataKuliah merupakan interface dasar yang digunakan oleh seluruh jenis mata kuliah.

Method getNama() digunakan untuk mengambil nama mata kuliah.

Method getSKS() digunakan untuk mengambil jumlah SKS mata kuliah.

Interface ini menjadi kontrak utama bagi seluruh jenis mata kuliah.

Praktikum.java

```
1  package matakuliah;
2
3  public interface Praktikum {
4
5      void alokasiAsistenLab();
6
7      void cekPeralatan();
8  }
```

Praktikum merupakan interface khusus yang digunakan untuk mata kuliah yang memiliki aktivitas laboratorium.

Method alokasiAsistenLab() digunakan untuk mengatur kebutuhan asisten laboratorium.

Method cekPeralatan() digunakan untuk memeriksa ketersediaan peralatan praktikum.

Pemisahan interface ini merupakan penerapan Interface Segregation Principle (ISP).

MataKuliahTeori.java

Class MataKuliahTeori merupakan implementasi dari interface MataKuliah.

Class ini digunakan untuk merepresentasikan mata kuliah teori yang tidak membutuhkan laboratorium.

Atribut nama digunakan untuk menyimpan nama mata kuliah.

Atribut sks digunakan untuk menyimpan jumlah SKS mata kuliah.

Method getName() dan getSKS() digunakan untuk mengambil data mata kuliah.

Karena tidak mengimplementasikan interface Praktikum, class ini terhindar dari method yang tidak diperlukan.

MataKuliahPraktikum.java

```
1  package matakuliah;
2
3  public class MataKuliahPraktikum implements MataKuliah, Praktikum {
4
5      private String nama;
6      private int sks;
7
8      public MataKuliahPraktikum(String nama, int sks) {
9          this.nama = nama;
10         this.sks = sks;
11     }
12
13     @Override
14     public String getName() {
15         return nama;
16     }
17
18     @Override
19     public int getSKS() {
20         return sks;
21     }
22
23     @Override
24     public void alokasiAsistenLab() {
25         System.out.println("Asisten lab dialokasikan");
26     }
27
28     @Override
29     public void cekPeralatan() {
30         System.out.println("Peralatan diperiksa");
31     }
32 }
```

Class MataKuliahPraktikum merupakan implementasi dari interface MataKuliah dan Praktikum.

Class ini digunakan untuk merepresentasikan mata kuliah yang memerlukan laboratorium.

Method alokasiAsistenLab() digunakan untuk mengalokasikan asisten laboratorium.

Method cekPeralatan() digunakan untuk memeriksa kesiapan peralatan praktikum.

Penerapan ini sesuai dengan Interface Segregation Principle (ISP) dan Liskov Substitution Principle (LSP).

MataKuliahKKN.java

```
src > matakuliah > J MataKuliahKKN.java > MataKuliahKKN
1  package matakuliah;
2
3  public class MataKuliahKKN implements MataKuliah {
4
5      private String nama;
6      private int sks;
7
8      public MataKuliahKKN(String nama, int sks) {
9          this.nama = nama;
10         this.sks = sks;
11     }
12
13     @Override
14     public String getNama() {
15         return nama;
16     }
17
18     @Override
19     public int getSKS() {
20         return sks;
21     }
22 }
```

Class MataKuliahKKN merupakan implementasi dari interface MataKuliah. Class ini digunakan untuk merepresentasikan mata kuliah Kuliah Kerja Nyata (KKN).

Karena tidak memiliki kebutuhan laboratorium, class ini tidak mengimplementasikan interface Praktikum.

Desain ini menghindari penggunaan UnsupportedOperationException yang sebelumnya menjadi masalah pada sistem lama.

KRSService.java

```

src > service > J KRSService.java > KRSService
1  package service;
2
3  import model.KRS;
4  import repository.KRSRepository;
5  import report.PDFGenerator;
6
7  public class KRSService {
8
9      private PrerequisiteValidator validator;
10     private KRSRepository repository;
11     private PDFGenerator pdfGenerator;
12
13     public KRSService(
14         PrerequisiteValidator validator,
15         KRSRepository repository,
16         PDFGenerator pdfGenerator) {
17
18         this.validator = validator;
19         this.repository = repository;
20         this.pdfGenerator = pdfGenerator;
21     }
22
23     public void prosesKRS(KRS krs) {
24
25         if (validator.validate(krs.getNim())) {
26
27             repository.save(krs);
28
29             pdfGenerator.generate(krs);
30         }
31     }
32 }

```

Class KRSService merupakan class utama yang mengoordinasikan proses pengelolaan KRS.

Atribut validator digunakan untuk melakukan validasi prasyarat mahasiswa.

Atribut repository digunakan untuk menyimpan data KRS ke database.

Atribut pdfGenerator digunakan untuk menghasilkan dokumen KRS dalam format PDF.

Constructor KRSService() digunakan untuk melakukan dependency injection terhadap objek yang dibutuhkan.

Method prosesKRS() digunakan untuk menjalankan alur pengolahan KRS mulai dari validasi, penyimpanan data, hingga pembuatan dokumen PDF.

Class ini menerapkan Dependency Inversion Principle (DIP) karena bergantung pada interface KRSRepository, bukan implementasi database tertentu.

Main.java

```

src > J Main.java > Main
1  import model.KRS;
2  import report.PDFGenerator;
3  import repository.KRSRepository;
4  import repository.NoSQLRepository;
5  import service.KRSService;
6  import service.PrerequisiteValidator;
7  import service.UKTCalculator;
8  import strategy.MBKMSStrategy;
9
10 public class Main {
11
12     Run | Debug
13     public static void main(String[] args) {
14
15         KRS krs = new KRS(nim: "105224014", totalSKS: 20);
16
17         UKTCalculator calculator =
18             new UKTCalculator(
19                 new MBKMStrategy());
20
21         double ukt =
22             calculator.calculate(
23                 krs.getTotalSKS());
24
25         System.out.println(
26             "Total UKT = " + ukt);
27
28         KRSRepository repository =
29             new NoSQLRepository();
30
31         KRSService service =
32             new KRSService(
33                 new PrerequisiteValidator(),
34                 repository,
35                 new PDFGenerator());
36
37         service.prosesKRS(krs);
38     }
39 }

```

Class Main merupakan titik awal eksekusi program.

Objek KRS dibuat untuk merepresentasikan data KRS mahasiswa.

Objek UKTCalculator dibuat dengan menggunakan implementasi MBKMStrategy sebagai strategi perhitungan UKT.

Method calculate() digunakan untuk menghitung biaya UKT mahasiswa.

Objek repository dibuat menggunakan implementasi NoSQLRepository untuk mensimulasikan migrasi database ke Cloud NoSQL.

Objek KRSService dibuat dengan dependency injection sehingga seluruh komponen dapat bekerja secara fleksibel.

Method prosesKRS() digunakan untuk menjalankan seluruh proses pengelolaan KRS mulai dari validasi hingga penyimpanan data.

IV. Kesimpulan

Berdasarkan hasil analisis dan perancangan ulang sistem SIAKAD, dapat disimpulkan bahwa sistem lama mengalami berbagai permasalahan arsitektur seperti God Class, Tight Coupling, serta pelanggaran terhadap prinsip-prinsip SOLID. Permasalahan tersebut menyebabkan sistem sulit dikembangkan, sulit dipelihara, dan rentan mengalami kerusakan ketika dilakukan perubahan pada salah satu fitur.

Melalui proses refactoring, sistem berhasil dirancang ulang dengan menerapkan prinsip Single Responsibility Principle (SRP), Open Closed Principle (OCP), Liskov Substitution Principle (LSP), Interface Segregation Principle (ISP), dan Dependency Inversion Principle (DIP). Setiap komponen sistem memiliki tanggung jawab yang lebih jelas sehingga meningkatkan modularitas dan keterbacaan kode.

Selain itu, penggunaan Strategy Pattern pada perhitungan UKT memungkinkan penambahan jalur mahasiswa baru seperti MBKM tanpa mengubah kode yang sudah ada. Penggunaan interface KRSRepository juga memungkinkan migrasi database dari MySQL ke Cloud NoSQL dengan perubahan yang minimal. Secara keseluruhan, sistem yang dihasilkan menjadi lebih fleksibel, mudah dikembangkan, serta lebih siap menghadapi kebutuhan akademik di masa mendatang.

V. Daftar Pustaka

Tim Asisten Laboratorium Komputer. (2026). Modul Praktikumj Pemrograman Berorientasi (PBO)

Modul 12 Program Studi Ilmu Komputer, Universitas Pertamina